Exercises

Exercise 7: Reproducible reports with R markdown

Read Chapter 8 to help you complete the questions in this exercise.

Before the session. You need the `rmarkdown` and `knitr` packages. Both normally come with RStudio, but please check before you arrive by running `library(rmarkdown)` and `library(knitr)` in the console. If either gives an error, install it with `install.packages("rmarkdown")`. See Appendix A of the Introduction to R book if you need more detail. If you run into real problems we will help you at the start of the session, but please try beforehand so we do not lose teaching time to installing software.

Everything you have produced so far has been code, and code is written for you. This exercise is about producing a *document*, written for somebody else, in which the code, the results and the writing all live in the same file and cannot drift apart. That is what 'reproducible' means: change the data, press one button, and every number, figure and table in the document updates itself.

You will build **one document** across the whole exercise, adding a layer at each question. Do not start a new file each time. By the end you will have a short report on the cardiac data and a template you can reuse. There is a lot of new vocabulary here, so take it slowly and knit often: knitting after every small change is the quickest way to see what each thing does, and the quickest way to find out which change broke something.

1. Open the RStudio Project you have been using for this course. Create a new R markdown document by going to `File` -> `New File` -> `R Markdown...`.

The first time you do this, RStudio may tell you it needs to install some packages. Let it, and wait for it to finish.

In the dialogue box that appears, type `Cardiac study report` in the Title box, put your name in the Author box, make sure **HTML** is selected on the left, and click OK. RStudio opens a new document that is already full of example content about cars. That is normal; we will delete it in a moment.

Before you change anything at all, save the file into your Project directory as `cardiac_report.Rmd` (`File` -> `Save As...`), then click the **Knit** button at the top of the editor. You can also press Ctrl+Shift+K, or Cmd+Shift+K on a Mac.

Do this before you understand any of it. If a document appears, your setup works, and anything that goes wrong later is something you did rather than something that was already broken.

2. Look at the document that appeared alongside the file that produced it. An R markdown file has only three kinds of content, and everything else is a variation on them:

- the **YAML header** at the very top, between the two lines of `---`, which controls the document as a whole
- **text**, written in a simple markup language called markdown
- **code chunks**, which look like this:

````
```{r}
mean(cardiac$age)
```
````

Find one of each in your file. Then delete everything below the first code chunk, the one called `setup`, leaving the YAML header and that chunk in place. Knit again to check you have an empty but working document. This is your starting point.

3. Now take control of the YAML header. At the moment it has a title, an author and a date. Add the four `output` lines shown below.

Two warnings, because this is where people lose ten minutes. Indentation in YAML matters, so copy the spacing exactly. And use spaces, never tabs.

```
---
title: "Cardiac study report"
author: "Your Name"
date: "20 August 2026"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
---
```

Add `toc: true` first and knit. Then add `toc_float: true` and knit again. Then `number_sections: true`. Adding them one at a time, and knitting after each, is how you see what each one actually does.

4. Time to write something. Underneath the setup chunk, write a short introduction to the cardiac study using markdown formatting only, with no code yet. Include:

- a heading, written as `# My heading`
- a second, smaller heading below it, written as `## My smaller heading`
- a sentence with a word in **bold**, written as `**bold**`, and a word in *italics*, written as `*italics*`
- a bulleted list of four of the variables in the dataset, each line starting with a dash and a space

That is enough to be going on with. If you want to add a link, a numbered list, a quotation or a table, Section 8.5.2 of the book has the syntax for all of them, and there is a summary on the R markdown cheat sheet under `Help -> Cheat Sheets`.

Knit, and put your source file and the output side by side. Notice that the headings you wrote have appeared in the table of contents on their own, because of the `toc: true` you added in Q3.

5. Now add some code. Insert a new code chunk using the **Insert** button at the top right of the editor and choosing R, or with the keyboard shortcut Ctrl+Alt+I (Cmd+Option+I on a Mac). An empty chunk appears.

Give the chunk a name by typing it straight after the `r`, so the first line reads ```` ```{r import} ````. Naming chunks costs nothing and makes them far easier to find when something goes wrong.

Inside the chunk, import `cardiacdata.txt` exactly as you did in **Exercise 3, Q5**: with the `read.table()` function, writing out the `header`, `sep`, `na.strings` and `stringsAsFactors` arguments, and assigning the result to `cardiac`.

Then insert a second chunk, name it `structure`, and put `str(cardiac)` in it.

Run each chunk in the editor with the small green arrow at its top right, then knit the whole document. Notice that knitting starts a completely fresh R session: anything you made earlier in the console does not exist when the document knits, which is exactly what makes the document reproducible. It also means the chunk that imports the data has to come before any chunk that uses it.

6. Add a figure. Insert another chunk, name it `chol-plot`, and use it to redraw the boxplot of total cholesterol by smoking status from **Exercise 5, Q10**. You will need the factor version of the smoking variable first, which you made in **Exercise 5, Q4**, so include that line in the chunk too.

Now give the figure a caption. Chunk options go inside the curly braces, after the chunk name, separated by commas. The first line of your chunk should read:

````
```{r chol-plot, fig.cap = 'Total cholesterol by smoking status.'}
```
````

Knit and find your caption underneath the plot. Then add `fig.width = 4` to the same line, knit again, and see what happens to the size of the text relative to the plot.

There are a great many other chunk options for figures, controlling height, alignment, resolution and so on. You do not need them today. Chapter 8 of the book lists the ones worth knowing.

7. Here is the question that turns a set of notes into a report. Ask yourself who is going to read this document. A colleague checking your working wants to see your code. A supervisor, a manager or an examiner almost certainly does not: they want the results, and the code is clutter. R markdown lets you decide chunk by chunk with the `echo` option.

a) Add `echo = FALSE` to your `chol-plot` chunk, alongside the caption you added in Q6, and knit. The plot is still there; the code that drew it is not.

b) Now do the same for the whole document rather than one chunk at a time. Go to your `setup` chunk at the top and put this line inside it:

```
knitr::opts_chunk$set(echo = FALSE)
```

Knit. Every chunk now hides its code, so you can delete the `echo = FALSE` you added in part (a) and it will still be hidden.

c) Finally, make one exception. Add `echo = TRUE` to the first line of your `structure` chunk, and knit. That one chunk shows its code while the rest do not. Setting a default for the document and then overriding it for individual chunks is how nearly every real report is put together.

8. So far every number in your document is inside a chunk. Numbers in your *writing* are the ones that go stale: you type '163 patients', the data changes, and now your report is wrong and nothing warns you.

R markdown fixes this with **inline code**. In the middle of an ordinary sentence you write a backtick, then the letter `r`, then some R code, then a closing backtick, and when you knit, the answer appears instead. Like this:

```
The study included `r nrow(cardiac)` patients.
```

which comes out as "The study included 163 patients."

Write a sentence of your own in the report that gives the number of patients and their mean age, using inline code for both rather than typing the numbers. Use `round()` on the mean so you do not get six decimal places.

Then test that it really works. Temporarily change your import chunk so it keeps only the first 100 patients, by adding this line at the end of the chunk:

```
cardiac <- head(cardiac, 100)
```

Knit, and watch your sentence update itself. Then delete that line and knit again.

9. Add a table. If you simply print a dataframe in a chunk you get the console output, monospaced and ugly. The `kable()` function from the `knitr` package formats it as a proper table instead.

Insert a chunk, name it `summary-table`, and start by recreating the `aggregate()` summary from **Exercise 4, Q9**: the mean of `age`, `systolic` and `tchol` for each smoking category. Assign it to an object called `smoke_summary`.

Knit and look at how ugly it is. Now put `knitr::kable(smoke_summary)` on the next line, knit again, and compare. Finally add `digits = 1` and a `caption` to the `kable()` call to tidy it up.

10. Not every figure in a report is one that R drew. You might need a diagram, a photograph, a map, or a figure somebody else made. Add the plot you exported in **Exercise 6, Q9**, `ex6_admissions.png`, to your report.

The simplest way needs no chunk at all. Type this straight into your text, changing the path to wherever your file actually is:

```
![Alcohol-related hospital admissions by council area.](output/ex6_admissions.png)
```

An exclamation mark, then the caption in square brackets, then the file path in round brackets. Knit and check the figure appears.

If you no longer have the file, download it here and save it into your `output` directory, the one you created in **Exercise 1, Q12**.

One thing that catches everybody out: the path is relative to where the `.Rmd` file is, not to your working directory. If the image does not appear, the path is almost always the reason.

11. If you have time, click the small arrow next to the **Knit** button and choose to knit to a Word document. Open it and compare it with the HTML version. Which would you send to a supervisor, and which would you put on a website? Keep both.

Two things worth knowing about, but not now

The visual editor. RStudio can show an R markdown document formatted as you type, more like a word processor, hiding the markdown syntax. You turn it on with the small compass icon at the top right of the editor. It is genuinely useful, especially for tables and links. We have deliberately left it until the end so that you learn what the markup actually is first, because when something goes wrong it is the markup you have to fix.

Quarto. Quarto is the successor to R markdown. It does the same job, works with languages other than R, and its syntax is so close that nearly everything in this exercise transfers unchanged. If you carry on using R after this course you will meet it. Start at quarto.org.

End of Exercise 7